

Equipe 1
Software Security
SI4 2025/2026

MALAY Nicolas

HITMI Younes

KARBAL Adam

GUERRAOUI Ismail

Sommaire

1. Introduction	4
2. Architecture générale du système.....	4
3. Tests de sécurité réalisés	5
3.1 Test de la politique de mot de passe	5
3.2 Test du verrouillage optimiste.....	6
3.3 Test de la réplication et de la haute disponibilité	6
3.4 Test du partage de notes	7
4. Conclusion	7

1. Introduction

Dans le cadre de ce projet, nous avons développé une application web permettant à des utilisateurs de créer, stocker, consulter et partager des notes textuelles personnelles. L'application repose sur une architecture client-serveur avec un stockage des données dans une base de données SQL répliquée afin d'assurer une haute disponibilité et une meilleure résilience face aux pannes.

L'un des objectifs majeurs du projet est la prise en compte de la sécurité dès la conception du système. En effet, les notes stockées peuvent contenir des informations personnelles ou sensibles, ce qui impose de fortes garanties en matière de confidentialité. Le système doit également empêcher tout accès non autorisé et assurer une gestion des droits entre les utilisateurs.

Ce rapport présente l'architecture générale du système ainsi que les tests de sécurité réalisés afin de démontrer la robustesse de l'implémentation et le respect des exigences de sécurité définies.

2. Architecture générale du système

L'application repose sur une architecture web moderne en trois couches, permettant une séparation claire des responsabilités et facilitant la sécurité, la maintenabilité et l'évolutivité du système.

Frontend

Le frontend est développé en Angular.

Il offre une interface utilisateur permettant aux utilisateurs de s'authentifier, de créer, consulter et gérer leurs notes, ainsi que de partager celles-ci avec d'autres utilisateurs.

Le frontend communique exclusivement avec le backend via des requêtes HTTP sécurisées. Plus précisément, le frontend interagit avec un point d'entrée unique (Port 8080) géré par un Load Balancer, qui masque la complexité de l'infrastructure backend.

et assure la redirection transparente des flux. Aucune logique sensible ni accès direct à la base de données n'est implémenté côté client.

Backend

Le backend repose sur un server node.js, chargé de la logique métier de l'application. Il gère :

- l'authentification des utilisateurs
- la validation et le traitement des requêtes
- la gestion des droits d'accès aux notes
- les opérations de création, modification et consultation des données.

Le backend intègre également un système de verrouillage optimiste (Locked Mode) s'appuyant sur un versioning SQL pour prévenir les conflits d'écriture lors de modifications concurrentes.

Des middlewares de sécurité sont utilisés afin de valider les entrées utilisateurs, contrôler les autorisations et prévenir les comportements malveillants.

Base de données

Les données sont stockées dans une base de données SQL PostgreSQL, assurant la persistance des données.

La base est configurée en réplication maître / esclave :

- le serveur maître accepte les opérations d'écriture ;
- le serveur esclave est utilisé en lecture seule et permet d'assurer une continuité de service en cas de défaillance du serveur principal.

Sécurité des échanges

Toutes les communications entre le frontend et le backend sont protégées par un mécanisme d'authentification basé sur des JSON Web Tokens.

Ce mécanisme permet de vérifier l'identité des utilisateurs à chaque requête et de limiter les risques d'accès non autorisé, de fuite de données ou d'attaques malveillantes. Le Load Balancer renforce cette sécurité en appliquant une politique CORS restrictive, n'autorisant que les requêtes provenant de l'origine de confiance du frontend (Port 4200).

Cette architecture globale contribue à renforcer la sécurité, la disponibilité et la fiabilité du système.

3. Tests de sécurité

Cette section présente les principaux tests de sécurité effectués afin de vérifier le bon fonctionnement des mécanismes de protection mis en place dans l'application.

3.1 Test de la politique de mot de passe

Objectif :

Vérifier que le système impose une politique de mot de passe robuste lors de la création d'un compte utilisateur.

Procédure :

Lors de l'inscription, l'utilisateur doit choisir :

- un nom d'utilisateur d'au moins 3 caractères ;
- un mot de passe d'au moins 8 caractères, contenant au minimum :
 - une lettre minuscule,
 - une lettre majuscule,
 - un chiffre,
 - un caractère spécial.

Résultat attendu :

Le système refuse toute inscription ne respectant pas ces contraintes.

Résultat obtenu :

Les tentatives d'inscription avec un mot de passe faible sont rejetées.

The image displays three sequential screenshots of a web registration form titled "Inscrivez-vous sur Secure Notes". The form includes a description: "L'application qu'il vous faut pour écrire et partager des notes entre amis de manière sécurisée". It features two input fields: one for a username (containing "ba") and one for a password (containing "baba200"). A blue "S'inscrire" button is positioned below the fields. Each screenshot shows a different validation error message in a red box at the bottom:

- First screenshot:** The error message is "'username' length must be at least 3 characters long".
- Second screenshot:** The error message is "Le mot de passe doit faire au moins 8 caractères".
- Third screenshot:** The error message is "Le mot de passe doit contenir une majuscule, une minuscule, un chiffre et un caractère spécial.".

Lorsque toutes les exigences sont respectées, le compte est créé avec succès.

```
Utilisateur créé! account-creation-page.component.ts:33
▼ {message: 'Utilisateur créé', token: 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOi...1MDJ9.wD74WiIUL_mFRvb-9pMKtmkXDkLcRmFr7D1elykMq-M', user: {...}}
  i
  message: "Utilisateur créé"
  token: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJMsInVzZXJuYW11IjIj..."
  ▼ user:
    id: 3
    username: "bab"
```

3.2 Test du verrouillage optimiste

Objectif :

Éviter les conflits lors de modifications concurrentes d'une même note.

Procédure :

Une note peut être modifiée à tour de rôle afin de vérifier que sa version est correctement incrémentée à chaque mise à jour. Deux utilisateurs (ou deux onglets) ouvrent la même note simultanément. L'utilisateur A enregistre une modification. L'utilisateur B tente ensuite d'enregistrer la sienne sans avoir rafraîchi la page.

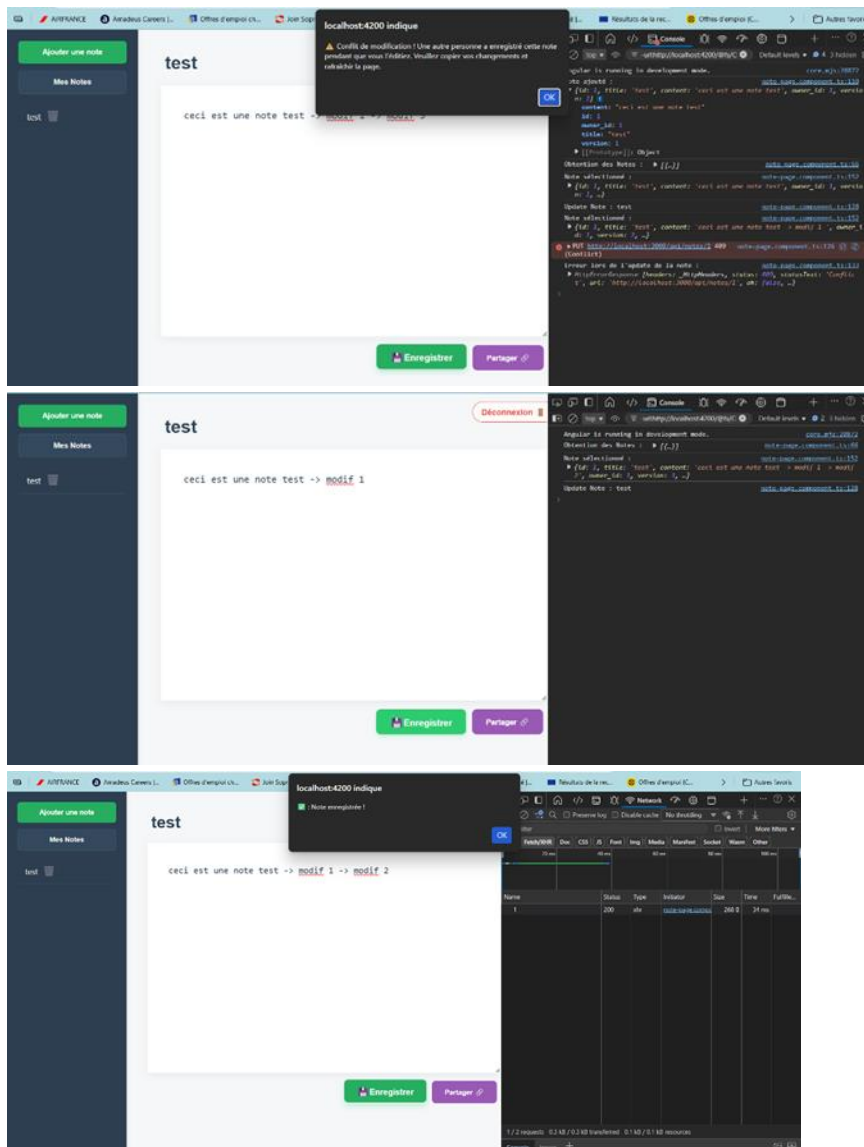
Résultat attendu :

La version de la note doit être mise à jour automatiquement à chaque modification, permettant de détecter les conflits d'édition.

Résultat obtenu :

La version de la note s'incrémente correctement après chaque modification, confirmant le bon fonctionnement du mécanisme de verrouillage optimiste.

Le système détecte que la version en base de données est supérieure à la version soumise par l'utilisateur B. La requête est rejetée avec une erreur 409 (Conflict) et un message d'alerte s'affiche sur l'interface, protégeant ainsi l'intégrité des données contre l'écrasement accidentel.



```
secure_notes_db=# SELECT * FROM notes;
 id | title |          content          | owner_id | version
-----+-----+-----+-----+-----
  1 | test | ceci est une note test |      1 |      1
(1 ligne)

secure_notes_db=# SELECT * FROM notes;
 id | title |          content          | owner_id | version
-----+-----+-----+-----+-----
  1 | test | ceci est une note test -> modif 1 -> modif 2 |      1 |      5
(1 ligne)
```

3.3 Test de la réplication et de la haute disponibilité

Objectif :

Vérifier que les données restent accessibles en cas de défaillance du serveur principal.

Procédure :

Deux serveurs PostgreSQL sont utilisés :

- un serveur maître écoutant sur le port 5432 ;
- un serveur esclave écoutant sur le port 5433.

La commande `SELECT pg_is_in_recovery()` est utilisée pour vérifier le rôle de chaque serveur. Une simulation d'arrêt du service sur le serveur maître est ensuite réalisée.

En complément du test de la base de données, nous simulons une panne du serveur applicatif primaire (Port 3000) en coupant son processus. Le frontend continue d'envoyer ses requêtes au Load Balancer (Port 8080).

Résultat attendu :

Le serveur maître doit retourner false et accepter les écritures.

Le serveur esclave doit retourner true et être accessible en lecture seule.

En cas de panne du maître, la lecture depuis l'esclave doit rester possible.

Résultat obtenu :

Les résultats observés correspondent au comportement attendu. La lecture des données reste fonctionnelle via le serveur esclave lorsque le serveur maître est indisponible. Le Load Balancer détecte l'erreur `ECONNREFUSED` sur le port 3000 et redirige instantanément la requête vers le serveur de secours (Port 3001) grâce à un mécanisme de bufferisation des flux. L'utilisateur ne perçoit aucune interruption de service.

Exemple :

On voit avec la capture suivante une simulation d'arrêt du service pgadmin :

 PNRPsvc		Protocole P...	Arrêté	LocalServic...
 PolicyAgent		Agent de st...	Arrêté	NetworkSer...
 postgresql-x64-17		postgresql-...	Arrêté	
 Power	1424	Alimentation	En cours d'...	DcomLaunch
 PrintNotify		Extensions ...	Arrêté	print

Voici ce qui se passe quand on essaye de se connecter au port 5432 et on obtiens une erreur connection refused :

```
secure_notes_db=# \q
PS C:\Users\youne> & "C:\Program Files\PostgreSQL\17\bin\psql.exe" -p 5432 -U postgres -d secure_notes_db
psql: erreur : la connexion au serveur sur « localhost » (:::1), port 5432 a échoué : Connection refused (0x0000274D/10061)
    Le serveur est-il actif sur cet hôte et accepte-t-il les connexions ?
la connexion au serveur sur « localhost » (127.0.0.1), port 5432 a échoué : Connection refused (0x0000274D/10061)
    Le serveur est-il actif sur cet hôte et accepte-t-il les connexions ?
```

Une fois que l'on essaye la même manipulation côté esclave sur le port 5433, la récupération des données en lecture seule fonctionne bien comme prévu :

```
PS C:\> & "C:\Program Files\PostgreSQL\17\bin\psql.exe" -p 5433 -U postgres -d secure_notes_db
psql (17.2)
Attention : l'encodage console (850) diffère de l'encodage Windows (1252).
    Les caractères 8 bits peuvent ne pas fonctionner correctement.
    Voir la section « Notes aux utilisateurs de Windows » de la page
    référence de psql pour les détails.
Saisissez « help » pour l'aide.

secure_notes_db=# SELECT * from notes;
 id | title | content | owner_id | version
-----+-----+-----+-----+-----
  1 | test | ceci est une note test -> modif 1 | 1 | 6
(1 ligne)
```

3.4 Test du partage de notes

Objectif :

Vérifier le respect des droits de partage entre utilisateurs.

Procédure :

Une note appartenant à un utilisateur est partagée avec un second utilisateur soit en lecture seule soit en lecture/écriture.

Résultat attendu :

En lecture seule, toute tentative de modification doit être refusée.

En lecture / écriture, la modification doit être autorisée.

Résultat obtenu :

Le comportement observé est conforme aux attentes : les restrictions d'accès sont correctement appliquées.

Exemple :

Lors de ce test, j'utilise le compte ladey déjà créé auparavant et je partage la note de test déjà créée avec le nouvel utilisateur adamostark :

```
Note partagé note-page.component.ts:167  
▶ {message: 'Permission accordée à ', username: 'adamostark'}  
>
```

Ce partage est fait en lecture seule. J'essaye donc de modifier la note. Voici le résultat :

test

ceci est une note test -> modif 1

 Vous consultez cette note en lecture seule.

```
▼ {message: 'Connexion réussie', token: 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOi0TY3fQ.6YUWDdu6I8CbB2CrWHEErHLhnakVTqr4rQ5mfl6Wfg', user: {...}} ⓘ  
  message: "Connexion réussie"  
  token: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOi0TY3fQ.6YUWDdu6I8CbB2CrWHEErHLhnakVTqr4rQ5mfl6Wfg"  
  ▶ user: {id: 2, username: 'adamostark'}  
  ▶ [[Prototype]]: Object  
Obtention des Notes : ▶ [{...}] note-page.component.ts:66  
Note sélectionné : note-page.component.ts:152  
▼ {id: 1, title: 'test', content: 'ceci est une note test -> modif 1', owner_id: 1, version: 6, ...} ⓘ  
  content: "ceci est une note test -> modif 1"  
  id: 1  
  owner_id: 1  
  title: "test"  
  user_can_edit: false  
  version: 6  
  ▶ [[Prototype]]: Object
```

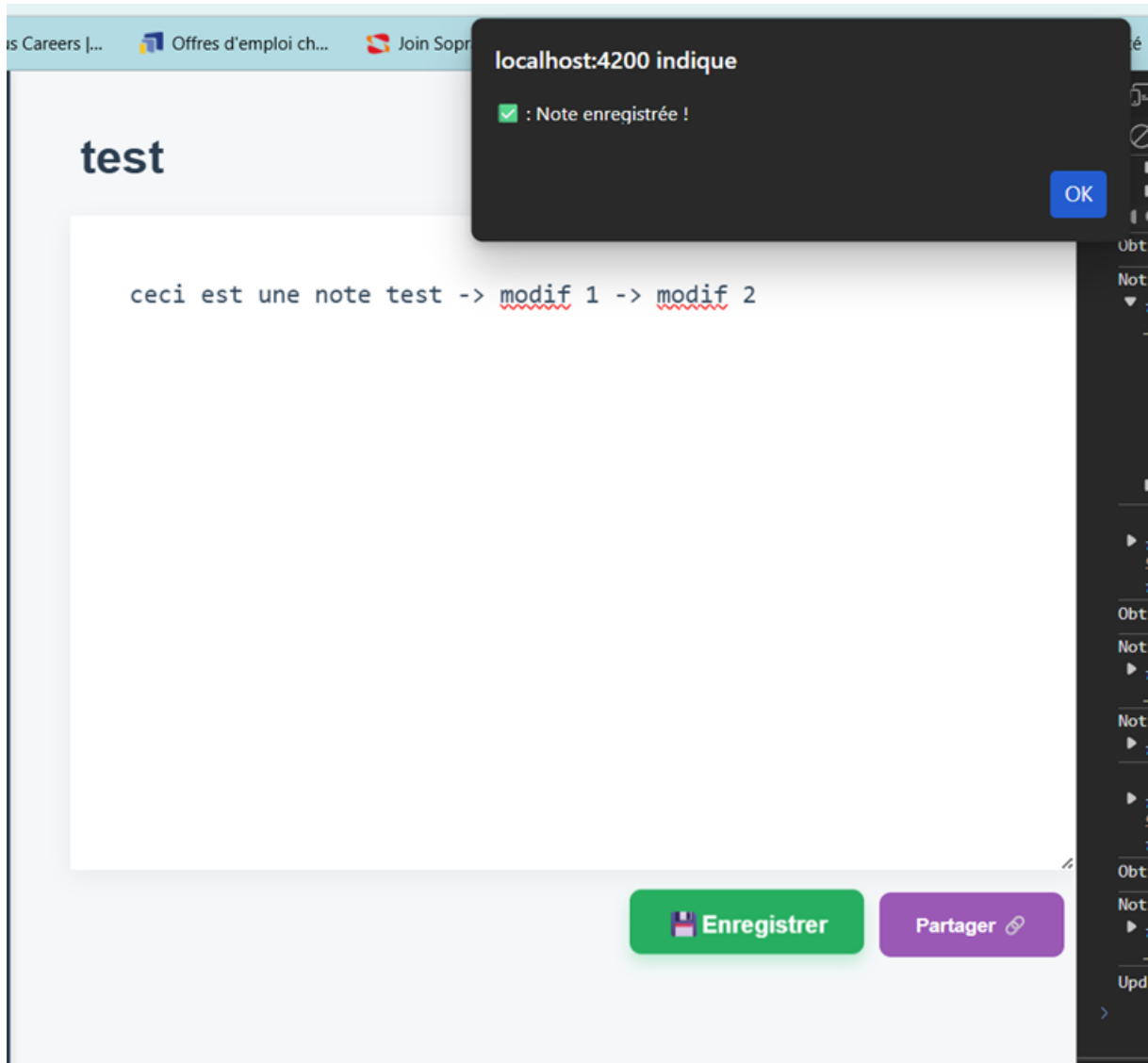
On ne peut rien y modifier. On essaye la même manipulation avec édition autorisée.
Voici le résultat :

Partager la note : "test"

Annuler

Partager

Écriture  



```
Note partagé note-page.component.ts:167
  ▶ {message: 'Permission accordée à ', username: 'adamostark'}
  home-page.component.ts:37
  ▶ {message: 'Connexion réussie', token: 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJkZ5fQ.kw008PNbLFtr4pNEar97hyFIWJj_tpSabIejhHe5q54', user: {...}}
Obtention des Notes : ▶ [{...}] note-page.component.ts:66
Note sélectionné : note-page.component.ts:152
  ▶ {id: 1, title: 'test', content: 'ceci est une note test -> modif 1', owner_id: 1, version: 6, ...}
Update Note : test note-page.component.ts:128
```

4. Conclusion

Ce projet nous a permis de mettre en pratique les notions de sécurité applicative vues en cours en les intégrant dès la conception de notre application. En mobilisant des solutions concrètes comme le verrouillage optimiste pour l'intégrité et un Load Balancer pour la disponibilité, nous avons pu démontrer la robustesse de notre architecture face aux scénarios de pannes et de conflits.

Cette réalisation nous a fait comprendre que la sécurité d'une application web ne se limite pas à l'authentification, mais englobe la gestion fine des accès (Confidentialité) et la continuité de service (Disponibilité). Les tests de sécurité effectués valident que les mécanismes implémentés répondent aux exigences fixées et garantissent un niveau de confiance élevé pour la protection des données sensibles des utilisateurs.